DESENVOLVIMENTO MOBILE · MÓDULO 13 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

Recursos do Dispositivo e IoT

Permissões, câmera, QR code, GPS e mapas, sensores, biometria, NFC, notificações, segundo plano, arquivos, áudio, Bluetooth LE com ESP32, serial USB com Arduino e MQTT

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Pedir permissões corretamente no Android e no iOS.
- Usar câmera, galeria, leitor de QR code, GPS, mapas e sensores.
- Proteger ações com biometria e ler etiquetas NFC.
- Exibir notificações locais e executar tarefas em segundo plano.
- Compartilhar arquivos, abrir links, tocar e gravar áudio.
- Comunicar-se com placas ESP32 e Arduino por Bluetooth LE, USB serial e MQTT.

Roteiro

Neste módulo

- Fundamentos: hardware, sensores e comunicação
- Como o Flutter acessa o hardware
- Permissões
- Câmera, galeria e QR code
- Localização e mapas
- Sensores e dados do aparelho
- Biometria e NFC
- Notificações e segundo plano
- Arquivos, links e áudio
- Bluetooth Low Energy com ESP32
- Serial USB com Arduino
- MQTT e escolha do protocolo
- Arquitetura e referência rápida
- Fechamento

Fundamentos: hardware, sensores e comunicação

Do mundo físico ao número



Conversor analógico-digital (ADC)

Mede a tensão em instantes regulares (**taxa de amostragem**) e a representa com um número de **bits**. Um ADC de 12 bits, como o do ESP32, distingue $2^{12} = 4096$ níveis.

Teorema da amostragem

Para reconstruir um sinal, amostre a mais que o dobro da maior frequência presente nele (Nyquist). Abaixo disso surgem frequências falsas (*aliasing*).

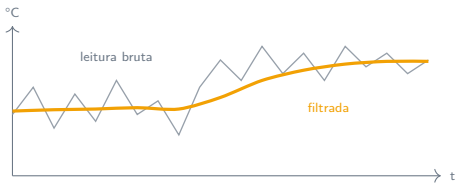
Sensores MEMS

Estruturas mecânicas microscópicas gravadas no silício. O **acelerômetro** mede aceleração (inclusive a da gravidade); o **giroscópio**, velocidade angular; o **magnetômetro**, o campo magnético; o **barômetro**, a pressão.

Atenção

Toda medida tem ruído, deriva e erro de calibração: exiba com a precisão que o sensor realmente tem.

Filtrando e combinando sensores



Média móvel exponencial

$y \leftarrow y + \alpha\,(x - y)$, com α entre 0 e 1. Pouco código, pouca memória: ótima para suavizar leituras.

Passa-baixas e passa-altas

Um filtro **passa-baixas** remove variações rápidas (ruído); um **passa-altas** remove variações lentas (deriva, gravidade). Quanto menor o α, mais suave e mais atrasada fica a curva.

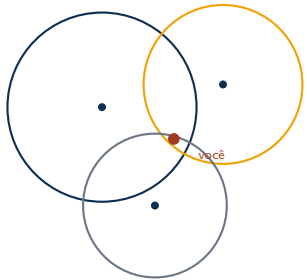
Fusão de sensores

Combinar sensores para compensar os defeitos de cada um. O giroscópio é preciso no curto prazo e deriva; o acelerômetro é ruidoso, mas não deriva. Juntos, dão a orientação do aparelho.

Dica

O sistema já entrega sensores fundidos, como o vetor de rotação. Prefira-os a reinventar a fusão.

Como o GPS descobre onde você está



cada satélite define uma esfera de distância; a interseção é a posição

GNSS e trilateração

Satélites transmitem a hora de seus relógios atômicos; o receptor calcula a distância pelo tempo de viagem do sinal. Com **quatro** satélites, resolve latitude, longitude, altitude e o erro do próprio relógio.

Várias constelações e ajuda da rede

Celulares usam GPS (EUA), GLONASS (Rússia), Galileo (UE) e BeiDou (China). O A-GPS baixa pela internet a posição dos satélites e reduz o primeiro acerto de minutos para segundos.

Atenção

Dentro de prédios o sinal some e o sistema recorre a Wi-Fi, torres e sensores. Ao ar livre, a precisão típica é de alguns metros.

Permissões e privacidade

Tipo	Concessão	Exemplos
Normal	automática, declarada no manifesto	internet, vibração
Em tempo de execução	o usuário decide num diálogo	câmera, localização, Bluetooth, notificações
Especial	tela própria nas configurações	alarmes exatos, sobrepor apps

Um pouco de história

O Android pede permissões em tempo de execução desde a versão 6.0 (2015). Vieram depois a permissão de uma só vez (11), a localização aproximada (12) e o seletor de fotos sem permissão (13).

Menor privilégio e minimização

Peça só o que o recurso exige, só quando ele for usado, e colete só os dados necessários. É boa engenharia e é exigência da LGPD (Módulo 15).

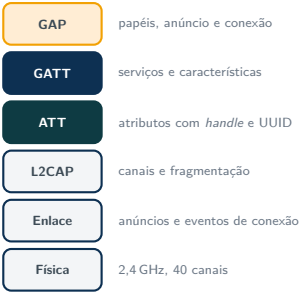
No iOS

Cada permissão precisa de um texto de justificativa no `Info.plist`; sem ele, o app é encerrado ao pedir o recurso.

Dica

Um ponto colorido na barra de status avisa quando câmera ou microfone estão em uso, no Android e no iOS.

Bluetooth Low Energy por dentro



Um pouco de história

O BLE surgiu no Bluetooth 4.0 (2010) para sensores alimentados por baterias pequenas. O Bluetooth 5 (2016) trouxe o dobro de velocidade e um modo de longo alcance.

Por que gasta tão pouco

O rádio fica quase sempre desligado. O periférico anuncia em 3 dos 40 canais e, conectado, acorda só a cada **intervalo de conexão** (de 7,5 ms a 4 s) para trocar alguns bytes.

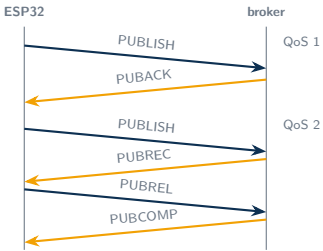
MTU

Tamanho máximo de um pacote ATT: 23 bytes por padrão (20 de dados úteis). Central e periférico podem negociar até 517 bytes.

Dica

Leituras frequentes: use **notify**. Mais dados por segundo: aumente o MTU e reduza o intervalo de conexão, sabendo que isso custa bateria.

MQTT por dentro



Um pouco de história

Andy Stanford-Clark (IBM) e Arlen Nipper criaram o MQTT em 1999 para monitorar oleodutos via satélite. Virou padrão OASIS em 2014; a versão 5.0 é de 2019.

Leve de propósito

Roda sobre TCP (porta 1883, ou 8883 com TLS) e o cabeçalho fixo tem só 2 bytes. Mensagens de *keep-alive* (PINGREQ e PINGRESP) mostram ao broker que a placa continua viva.

Sessão persistente

Com sessão persistente, o broker guarda as assinaturas e as mensagens de QoS 1 e 2 enquanto o cliente está desconectado e as entrega na volta.

Comunicação serial e UART



UART

Comunicação serial **assíncrona**: não há fio de relógio. Os dois lados combinam a velocidade (*baud rate*, como 115 200) e o formato, quase sempre **8N1**: 8 bits de dados, sem paridade, 1 bit de parada.

Ligação

TX de um lado no RX do outro, e terra comum. Atenção aos níveis: o ESP32 trabalha em 3,3 V, o Arduino Uno em 5 V e o RS-232 em ± 12 V.

Do USB à serial

A placa aparece no celular como dispositivo USB; um conversor (CH340, CP2102, FTDI) ou o próprio microcontrolador traduz para UART.

Monte um protocolo

A serial só entrega bytes. Defina mensagens com delimitador (como `\n`) e, se o ambiente for ruidoso, uma soma de verificação.

NFC: comunicação por proximidade

NFC

Comunicação a 13,56 MHz por acoplamento indutivo, com alcance de poucos centímetros. A etiqueta passiva não tem bateria: é alimentada pelo campo do próprio leitor.

Modo	Uso
Leitura e escrita	etiquetas com URL, texto ou identificador
Emulação de cartão	o celular se passa por cartão (pagamentos, crachás)

NDEF

Formato padrão das mensagens gravadas em etiquetas: uma lista de registros, cada um com tipo (URI, texto, MIME) e conteúdo.

No LabMonitor

Uma etiqueta colada na bancada guarda `https://labmonitor.app/bancada/b3`: encostar o celular abre a tela da bancada por link profundo (Módulo 07).

Atenção

A distância curta é uma proteção, mas não substitui a autenticação: qualquer um pode ler ou copiar uma etiqueta sem proteção.

Energia: o recurso mais escasso

Consumidor	Por que gasta	Como economizar
Tela	maior consumo do aparelho	tema escuro em telas OLED; nada de manter a tela ligada sem motivo
GPS	receptor ligado e cálculos contínuos	localização aproximada, intervalos maiores, cercas virtuais
Rede móvel	rádio de alta potência e tempo de cauda	agrupar requisições, *push* em vez de *polling*
CPU	despertares frequentes	agendadores do sistema em vez de temporizadores próprios
Bluetooth	varreduras longas	filtros por serviço e varreduras curtas

Um pouco de história

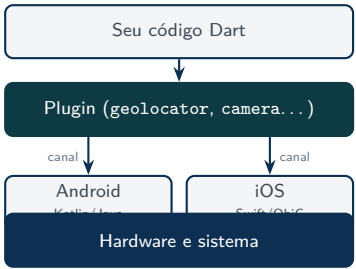
O Android criou o modo Doze (6.0, 2015), limitou serviços em segundo plano (8.0, 2017) e passou a classificar apps por frequência de uso (9.0, 2018). O iOS sempre restringiu o segundo plano a modos específicos.

Regra geral

Deixe o sistema decidir **quando** executar: WorkManager no Android, `BGTaskScheduler` no iOS. Ele agrupa o trabalho de vários apps e acorda o aparelho menos vezes.

Como o Flutter acessa o hardware

Plugins: Dart por cima, nativo por baixo



Plugin

Pacote com uma API em Dart e implementações nativas para cada plataforma, ligadas por **canais de plataforma** (Módulo 19 mostra como criar o seu).

Escolhendo um plugin no pub.dev

Prefira selo **Flutter Favorite** ou publicador verificado (`flutter.dev`, `fluttercommunity.dev`); confira plataformas suportadas, data da última versão, *issues* abertas e licença.

Atenção

Recurso de hardware se testa em **aparelho real**. Emuladores simulam GPS e sensores, mas não Bluetooth, NFC ou USB.

Permissões

Declarar nas duas plataformas

android/app/src/main/AndroidManifest.xml

```
<uses-permission
    android:name="android.permission.CAMERA" />
<uses-permission android:name=
    "android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name=
    "android.permission.POST_NOTIFICATIONS" />
```

ios/Runner/Info.plist

```
<key>NSCameraUsageDescription</key>
<string>Fotografar equipamentos e ler QR codes.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Confirmar presença no laboratório.</string>
```

Duas etapas

Declarar: manifesto (Android) e `Info.plist` (iOS).
Pedir: em tempo de execução, no momento em que o recurso é usado.

Rejeição na App Store

Falta de texto de uso no `Info.plist` derruba o app no iOS; texto vago (“para funcionar”) reprova na revisão. Explique o benefício ao usuário.

Pedindo com `permission_handler`

utils/permissoes.dart

```dart
Future<bool> garantir(Permission p,
    Future<bool> Function() explicar) async {
  var status = await p.status;
  if (status.isGranted || status.isLimited) return true;

  if (await p.shouldShowRequestRationale &&
      !await explicar()) return false;     // diálogo seu

  status = await p.request();
  if (status.isPermanentlyDenied) {
    await openAppSettings();               // só o usuário
    return false;
  }
  return status.isGranted || status.isLimited;
}

// várias de uma vez
final r = await [Permission.bluetoothScan,
    Permission.bluetoothConnect].request();
```

Estados

`granted`, `denied`, `permanentlyDenied`, `limited` (acesso parcial, como só algumas fotos) e `restricted` (controle parental).

iOS

O `permission_handler` exige ativar cada permissão no `ios/Podfile` (macros `PERMISSION_CAMERA=1` etc.).

Dica

Peça no contexto: ao tocar em "Ler QR code", não na abertura do app.

Câmera, galeria e QR code

Foto rápida com `image_picker`

Foto do equipamento

```dart
final _picker = ImagePicker();

Future<XFile?> fotografar() => _picker.pickImage(
      source: ImageSource.camera,
      maxWidth: 1600,          // reduz o arquivo
      imageQuality: 80,
    );

Future<List<XFile>> escolherDaGaleria() =>
    _picker.pickMultiImage(limit: 5);

// Android: o sistema pode encerrar o app
// enquanto a câmera está aberta
Future<void> recuperarPerdidos() async {
  final r = await _picker.retrieveLostData();
  if (!r.isEmpty) salvar(r.files!);
}
```

Quando usar

Para tirar uma foto ou escolher imagens usando o app de câmera e o seletor do próprio sistema. Também grava vídeo com `pickVideo`.

Galeria sem permissão

No Android, o **Photo Picker** do sistema dispensa permissão de armazenamento: o usuário escolhe e o app recebe só aquelas fotos.

Dica

`XFile` vem do pacote `cross_file`: funciona também na web.

Câmera dentro do app com `camera`

Tela de captura (trechos do State)

```dart
CameraController? _ctrl;

Future<void> _iniciar() async {
  final cams = await availableCameras();
  final traseira = cams.firstWhere(
      (c) => c.lensDirection == CameraLensDirection.back);
  final ctrl = CameraController(traseira,
      ResolutionPreset.high, enableAudio: false);
  await ctrl.initialize();
  if (mounted) setState(() => _ctrl = ctrl);
}

@override
void didChangeAppLifecycleState(AppLifecycleState s) {
  if (s == AppLifecycleState.inactive) _ctrl?.dispose();
  if (s == AppLifecycleState.resumed) _iniciar();
}

// State com o mixin WidgetsBindingObserver
// build: CameraPreview(_ctrl!)
// botão: final foto = await _ctrl!.takePicture();
```

Quando usar

Câmera como parte da interface: moldura sobre a imagem, fotos em sequência, quadros para visão computacional (`startImageStream`).

Ciclo de vida

Libere a câmera quando o app sai de cena e reabra ao voltar; outro app pode precisar dela.

QR code e código de barras

Lendo a etiqueta do equipamento

```
final _scanner = MobileScannerController(
    formats: const [BarcodeFormat.qrCode]);

@override
Widget build(BuildContext context) => MobileScanner(
  controller: _scanner,
  onDetect: (captura) {
    final valor = captura.barcodes.first.rawValue;
    if (valor == null) return;
    final uri = Uri.tryParse(valor);
    if (uri?.scheme == 'labmonitor') {
      _scanner.stop();
      context.go('/equipamentos/${uri!.pathSegments.last}');
    }
  },
);
```

Gerando a etiqueta (qr_flutter)

```
QrImageView(data: 'labmonitor://eq/42', size: 180)
```

mobile_scanner

Lê QR code, EAN, Code 128 e outros formatos usando ML Kit (Android) e Vision (iOS), sem internet.

Atenção

Valide o conteúdo lido: um QR code pode conter qualquer texto. Aceite só o seu esquema e IDs válidos.

Dica

Pare o leitor depois da primeira leitura válida para não navegar várias vezes.

Localização e mapas

GPS com `geolocator`

data/services/localizacao_service.dart

```
Future<Position> posicaoAtual() async {
  if (!await Geolocator.isLocationServiceEnabled()) {
    throw const LocalizacaoDesligada();
  }
  var p = await Geolocator.checkPermission();
  if (p == LocationPermission.denied) {
    p = await Geolocator.requestPermission();
  }
  if (p == LocationPermission.deniedForever) {
    throw const PermissaoNegada();   // openAppSettings()
  }
  return Geolocator.getCurrentPosition();
}

Stream<Position> acompanhar() =>
    Geolocator.getPositionStream(locationSettings:
        const LocationSettings(distanceFilter: 10));

bool noLab(Position p, Lab l) => Geolocator.distanceBetween(
    p.latitude, p.longitude, l.lat, l.lng) < 50;
```

Precisão e bateria

`LocationSettings` define a precisão (`high` usa GPS e gasta mais) e o `distanceFilter` (emitir só após andar N metros).

Check-in

`noLab` é uma cerca virtual (*geofence*) simples: presença só a menos de 50 m.

Atenção

Localização em segundo plano exige permissão extra e justificativa nas lojas.

Mapas

google_maps_flutter

```
GoogleMap(
  initialCameraPosition: const CameraPosition(
      target: LatLng(-2.5560, -44.3080), zoom: 17),
  myLocationEnabled: true,
  markers: {
    const Marker(
      markerId: MarkerId('lab-3'),
      position: LatLng(-2.5560, -44.3080),
      infoWindow: InfoWindow(title: 'Lab. 3'),
    ),
  },
)
```

Atenção

Exige chave da Google Maps Platform, restrita ao seu app.

flutter_map + OpenStreetMap

```
FlutterMap(
  options: const MapOptions(
      initialCenter: LatLng(-2.5560, -44.3080),
      initialZoom: 17),
  children: [
    TileLayer(
      urlTemplate: 'https://tile.openstreetmap'
          '.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'br.edu.labmonitor'),
    const MarkerLayer(markers: [Marker(
      point: LatLng(-2.5560, -44.3080),
      child: Icon(Icons.science))]),
  ],
)
```

Dica

Sem chave; mostre a atribuição do OSM e, com muito tráfego, use outro provedor.

Sensores e dados do aparelho

Sensores de movimento com `sensors_plus`

Agitar para atualizar

```dart
StreamSubscription<UserAccelerometerEvent>? _sub;
DateTime _ultimo = DateTime(0);

void detectarAgitar(VoidCallback aoAgitar) {
  _sub = userAccelerometerEventStream(
    samplingPeriod: SensorInterval.gameInterval,
  ).listen((e) {
    final a = sqrt(e.x * e.x + e.y * e.y + e.z * e.z);
    final agora = DateTime.now();
    if (a > 15 && agora.difference(_ultimo).inSeconds > 1) {
      _ultimo = agora;
      aoAgitar();
    }
  });
}

void parar() => _sub?.cancel();
```

Stream	Mede
`accelerometer...`	aceleração com gravidade (m/s^2)
`userAccelerometer...`	aceleração sem gravidade
`gyroscope...`	rotação (rad/s)
`magnetometer...`	campo magnético (μT)
`barometer...`	pressão (hPa)

Atenção

Sensores gastam bateria: cancele a assinatura quando a tela sai e use a menor frequência que resolve.

Bateria, rede e dados do aparelho

Pacotes "plus" da comunidade

```dart
// battery_plus
final bateria = Battery();
final nivel = await bateria.batteryLevel;          // 0..100
bateria.onBatteryStateChanged.listen((s) =>
    carregando = s == BatteryState.charging);

// connectivity_plus
Connectivity().onConnectivityChanged.listen((tipos) =>
    offline = tipos.contains(ConnectivityResult.none));

// device_info_plus e package_info_plus
final info = await DeviceInfoPlugin().androidInfo;
log('${info.model} - Android SDK ${info.version.sdkInt}');
final app = await PackageInfo.fromPlatform();
log('LabMonitor ${app.version}+${app.buildNumber}');

// wakelock_plus: tela sempre ligada no modo painel
WakelockPlus.enable();
```

Usos no LabMonitor

- Reduzir a frequência de leitura com bateria baixa.
- Mostrar a faixa “sem conexão”.
- Enviar modelo, sistema e versão do app nos relatórios de erro.
- Manter o tablet do laboratório com a tela acesa.

Dica

Estes pacotes são mantidos pela **Flutter Community** (`plus_plugins`).

Biometria e NFC

Biometria com `local_auth`

Confirmar uma ação sensível (local_auth 3)

```dart
final _auth = LocalAuthentication();

Future<bool> confirmarIdentidade() async {
  final pode = await _auth.canCheckBiometrics ||
      await _auth.isDeviceSupported();
  if (!pode) return false;
  try {
    return await _auth.authenticate(
      localizedReason: 'Confirme para liberar a bancada',
      biometricOnly: false,       // aceita PIN/padrão
      persistAcrossBackgrounding: true,
    );
  } on LocalAuthException {
    return false;                 // cancelado, bloqueado...
  }
}
```

Configuração

Android: a `MainActivity` deve estender `FlutterFragmentActivity`. iOS: `NSFaceIDUsageDescription` no `Info.plist`.

O que a biometria prova

Só que o dono do aparelho está presente. Ela **desbloqueia** um segredo local (Módulo 10); não substitui o login no servidor.

NFC: etiquetas nas bancadas

Check-in por aproximação (flutter_nfc_kit)

```dart
Future<String?> lerEtiqueta() async {
  final disp = await FlutterNfcKit.nfcAvailability;
  if (disp != NFCAvailability.available) return null;
  try {
    final tag = await FlutterNfcKit.poll(
      timeout: const Duration(seconds: 15),
      iosAlertMessage: 'Aproxime da etiqueta da bancada',
    );
    return tag.id;            // identificador único da tag
  } finally {
    await FlutterNfcKit.finish(iosAlertMessage: 'Pronto!');
  }
}
```

Dica

Para ler ou gravar dados na tag (uma URL, um texto), use `readNDEFRecords` e `writeNDEFRecords`.

NFC

Comunicação a poucos centímetros. Etiquetas NTAG custam centavos e não precisam de bateria: ótimas para identificar bancadas e equipamentos.

Configuração

Android: permissão `android.permission.NFC`. iOS: capacidade *NFC Tag Reading* no Xcode e `NFCReaderUsageDescription`; o iOS mostra uma folha de leitura própria.

Notificações e segundo plano

Notificações locais

flutter_local_notifications (versão 20+)

```dart
final _notif = FlutterLocalNotificationsPlugin();
const _android =
      AndroidInitializationSettings('@mipmap/ic_launcher');
Future<void> iniciar() async {
  await _notif.initialize(
    settings: const InitializationSettings(android: _android,
        iOS: DarwinInitializationSettings()),
    onDidReceiveNotificationResponse: (r) =>
        router.go(r.payload ?? '/'));
  await _notif.resolvePlatformSpecificImplementation<
          AndroidFlutterLocalNotificationsPlugin>()
      ?.requestNotificationsPermission();   // Android 13+
}

const _alertas = NotificationDetails(
  android: AndroidNotificationDetails('alertas', 'Alertas',
      importance: Importance.high, priority: Priority.high));
Future<void> alertar(int b, double t) => _notif.show(
    id: b, title: 'Bancada $b', body: 'Temperatura: $t °C',
    notificationDetails: _alertas, payload: '/bancadas/$b');
```

Canais (Android 8+)

Cada notificação pertence a um canal (`'alertas'`); o usuário pode silenciar um canal sem perder os outros.

Dica

`zonedSchedule` agenda lembretes (pacote `timezone`).

Atenção

Desde a versão 20 os parâmetros são nomeados; exemplos antigos não compilam.

Trabalho em segundo plano

Sincronização periódica (workmanager)

```dart
@pragma('vm:entry-point')
void despachante() {
  Workmanager().executeTask((tarefa, dados) async {
    WidgetsFlutterBinding.ensureInitialized();
    await sincronizarLeituras();          // Módulo 11
    return true;                          // sucesso
  });
}

Future<void> agendar() async {
  await Workmanager().initialize(despachante);
  await Workmanager().registerPeriodicTask(
    'sync-leituras', 'sincronizar',
    frequency: const Duration(minutes: 15),
    constraints: Constraints(
        networkType: NetworkType.connected),
  );
}
```

Necessidade	Solução
tarefa adiável	WorkManager (Android, mín. 15 min); BGTaskScheduler (iOS, quando o sistema quiser)
tarefa contínua	*foreground service* com notificação fixa (`flutter_foreground_task`)
BLE em segundo plano	iOS: modo `bluetooth-central`
aviso do servidor	push com FCM (Módulo 12)

Atenção

O sistema decide quando rodar e encerra apps que gastam bateria. No Android 14+, todo *foreground service* declara um tipo (`connectedDevice`, `location`...).

Arquivos, links e áudio

Compartilhar, escolher arquivos e abrir links

share_plus, file_picker e url_launcher

```dart
// enviar o relatório por qualquer app instalado
await SharePlus.instance.share(ShareParams(
  text: 'Relatório do Lab. 3 - outubro',
  files: [XFile(caminhoCsv)],
));

// importar uma planilha de equipamentos
final r = await FilePicker.platform.pickFiles(
    type: FileType.custom, allowedExtensions: ['csv']);
final arquivo = r?.files.single.path;

// telefone, e-mail, mapa e navegador
await launchUrl(Uri.parse('tel:+559832180000'));
await launchUrl(Uri.parse('mailto:lab3@ifma.edu.br'));
await launchUrl(Uri.parse('https://labmonitor.dev'),
    mode: LaunchMode.externalApplication);
```

Deep links

O `go_router` abre a rota certa quando o app é aberto por um link. Com **App Links** (Android) e **Universal Links** (iOS), `https://labmonitor.dev/eq/42` abre o app em vez do navegador.

Atenção

Android 11+: declare em `<queries>` os esquemas (`tel`, `mailto`, `https`) usados com `canLaunchUrl`.

Dica

A API antiga `Share.shareXFiles` foi substituída por `SharePlus.instance.share`.

Vibração, som, voz e microfone

Alarme de temperatura

```dart
HapticFeedback.heavyImpact();        // já vem no Flutter

final player = AudioPlayer();        // just_audio
await player.setAsset('assets/sons/alarme.mp3');
await player.setLoopMode(LoopMode.one);
player.play();

final tts = FlutterTts();            // flutter_tts
await tts.setLanguage('pt-BR');
await tts.speak('Temperatura alta na bancada três');

final gravador = AudioRecorder();    // record
if (await gravador.hasPermission()) {
  await gravador.start(const RecordConfig(), path: destino);
}
final arquivo = await gravador.stop();
```

Pacote	Para quê
`HapticFeedback`	toque tátil curto
`vibration`	padrões longos de vibração
`just_audio`	tocar áudio local ou remoto
`flutter_tts`	texto em fala
`speech_to_text`	fala em texto
`record`	gravar do microfone

Atenção

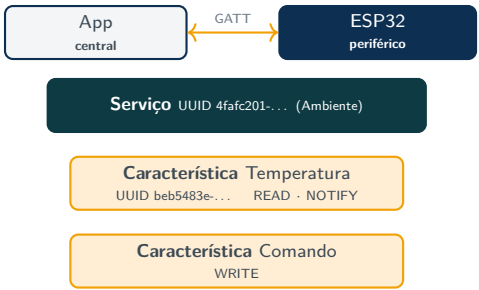
O microfone exige permissão nas duas plataformas (`RECORD_AUDIO` no Android).

Dica

Alertas importantes: som, vibração **e** texto.

Bluetooth Low Energy com ESP32

Conceitos de BLE



- **Central** procura e conecta; **periférico** anuncia (*advertising*) e serve dados.
- **GATT**: dados organizados em **serviços** e **características**, cada um com um UUID.
- Operações: **read**, **write** e **notify** (o periférico envia quando o valor muda).
- **MTU**: tamanho máximo de cada pacote (23 a 512 bytes).

Atenção

BLE é diferente do Bluetooth clássico: módulos HC-05/HC-06 do Arduino **não** são BLE. Use ESP32, HM-10 ou nRF52.

Firmware do ESP32 (Arduino)

labmonitor_ble.ino -- configuração

```
#include <BLEDevice.h>
#include <BLE2902.h>
#define SVC_UUID  "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define TEMP_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
BLECharacteristic *temp;
void setup() {
  BLEDevice::init("LabMonitor-B3");
  BLEServer *srv = BLEDevice::createServer();
  BLEService *svc = srv->createService(SVC_UUID);
  temp = svc->createCharacteristic(TEMP_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY);
  temp->addDescriptor(new BLE2902());   // CCCD
  svc->start();
  BLEDevice::getAdvertising()->addServiceUUID(SVC_UUID);
  BLEDevice::startAdvertising();
}
```

-- laço principal

```
void loop() {
  // centésimos de grau: 24,87 °C -> 2487
  int16_t c = round(lerTemperatura() * 100);
  temp->setValue((uint8_t*)&c, sizeof(c));
  temp->notify();
  delay(1000);
}
```

Formato binário

`int16_t` em centésimos de grau: 2 bytes, *little-endian*.

Contrato

Documente UUIDs, escala e ordem dos bytes; gere UUIDs próprios.

Permissões e licença do `flutter_blue_plus`

AndroidManifest.xml

```
<uses-permission android:name=
    "android.permission.BLUETOOTH_SCAN"
    android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name=
    "android.permission.BLUETOOTH_CONNECT" />
<!-- Android 11 ou anterior -->
<uses-permission android:name=
    "android.permission.ACCESS_FINE_LOCATION"
    android:maxSdkVersion="30" />
```

Info.plist

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Conectar aos sensores das bancadas.</string>
```

Licença do pacote

Desde a versão 2.0 (2025) o `flutter_blue_plus` usa licença própria, que já mudou mais de uma vez: uso pessoal, educacional e sem fins lucrativos é livre; uso comercial pode exigir licença paga, e o pacote envia telemetria de licença no *build*. Leia o `LICENSE` da versão que adotar.

Alternativas

`universal_ble` e `bluetooth_low_energy`; compare licença, plataformas e manutenção.

Procurando o ESP32

data/services/ble_service.dart

```dart
final svcUuid = Guid('4fafc201-1fb5-459e-8fcc-c5c9c331914b');
final tempUuid = Guid('beb5483e-36e1-4688-b7f5-ea07361b26a8');

Future<List<ScanResult>> procurar() async {
  await FlutterBluePlus.adapterState
      .where((s) => s == BluetoothAdapterState.on).first;
  final achados = <DeviceIdentifier, ScanResult>{};
  final sub = FlutterBluePlus.onScanResults.listen((rs) {
    for (final r in rs) achados[r.device.remoteId] = r;
  });
  FlutterBluePlus.cancelWhenScanComplete(sub);
  await FlutterBluePlus.startScan(
      withServices: [svcUuid],
      timeout: const Duration(seconds: 10));
  await FlutterBluePlus.isScanning
      .where((v) => v == false).first;
  return achados.values.toList();
}
```

ScanResult

Traz o dispositivo, o nome anunciado (`advertisementData.advName`) e o **RSSI** (força do sinal, que indica a proximidade).

Dica

O filtro `withServices` faz o sistema entregar só os seus dispositivos e economiza bateria.

Atenção

Peça `bluetoothScan` e `bluetoothConnect` antes de procurar e trate o Bluetooth desligado.

Conectando e recebendo a temperatura

Notify vira Stream<double>

```
Stream<double> temperatura(BluetoothDevice d) async* {
  await d.connect();
  final servicos = await d.discoverServices();
  final c = servicos
      .firstWhere((s) => s.uuid == svcUuid)
      .characteristics
      .firstWhere((c) => c.uuid == tempUuid);
  await c.setNotifyValue(true);
  yield* c.onValueReceived.map((bytes) {
    final v = ByteData.sublistView(Uint8List.fromList(bytes))
        .getInt16(0, Endian.little);
    return v / 100;                  // centésimos -> °C
  });
}

// enviar um comando
await comando.write(utf8.encode('LED=1'));
// ao sair: await d.disconnect();
```

Fluxo

Conectar → descobrir serviços → achar a característica → ativar notify → converter os bytes.

Desconexões

Observe `d.connectionState`; ao reconectar, descubra os serviços de novo. Use `d.cancelWhenDisconnected(sub)` para não acumular assinaturas.

Dica

O `Stream` entra no view model como qualquer outro (Módulos 06 e 08).

Serial USB com Arduino

Arduino pelo cabo USB (Android)

sensor_serial.ino

```
void setup() {
  Serial.begin(115200);
  pinMode(13, OUTPUT);
}
void loop() {
  float t = analogRead(A0)
      * (5.0 / 1023.0) * 100;  // LM35
  Serial.print("T=");
  Serial.println(t, 2);
  if (Serial.available()) {
    String cmd =
        Serial.readStringUntil('\n');
    digitalWrite(13,
        cmd == "LED=1" ? HIGH : LOW);
  }
  delay(1000);
}
```

Lendo linhas com usb_serial

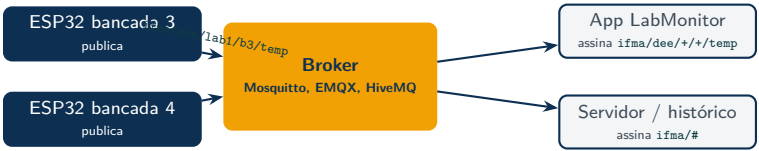
```
Future<Stream<String>> abrirSerial() async {
  final dispositivos = await UsbSerial.listDevices();
  final porta = await dispositivos.first.create();
  if (porta == null || !await porta.open()) {
    throw const SerialIndisponivel();
  }
  await porta.setDTR(true);
  await porta.setRTS(true);
  await porta.setPortParameters(115200, UsbPort.DATABITS_8,
      UsbPort.STOPBITS_1, UsbPort.PARITY_NONE);
  return porta.inputStream!
      .cast<List<int>>()
      .transform(utf8.decoder)
      .transform(const LineSplitter());      // "T=24.87"
}
```

Atenção

Só no Android, com cabo **OTG**; o sistema pede autorização ao conectar. O iOS não permite serial USB genérica.

MQTT e escolha do protocolo

MQTT: publicar e assinar



Tópicos

Hierarquia com /. Curingas: + (um nível) e # (todos os níveis abaixo).

QoS

0: no máximo uma vez. **1**: pelo menos uma vez. **2**: exatamente uma vez.

Retained / LWT

Retained guarda o último valor do tópico. *Last Will* avisa quando a placa cai.

MQTT no app

data/services/mqtt_service.dart (mqtt_client)

```dart
Future<Stream<(String, double)>> leituras(String f) async {
  final c = MqttServerClient.withPort(
      'broker.labmonitor.ifma.edu.br', clienteId, 8883)
    ..secure = true                          // TLS
    ..keepAlivePeriod = 30
    ..autoReconnect = true;
  await c.connect(usuario, senha);
  c.subscribe(f, MqttQos.atLeastOnce);
  return c.updates!.expand((lote) => lote).map((m) {
    final pub = m.payload as MqttPublishMessage;
    final txt = MqttPublishPayload
        .bytesToStringAsString(pub.payload.message);
    return (m.topic, double.parse(txt));
  });
}
```

ESP32 (PubSubClient)

```arduino
mqtt.setServer(BROKER, 1883);
mqtt.connect("esp32-b3", USER, PASS,
  "ifma/dee/lab1/b3/status", 1, true,
  "offline");                          // LWT
mqtt.publish("ifma/dee/lab1/b3/temp",
  String(t, 2).c_str(), true); // retained
```

Segurança

Broker público só para testes. Em produção: TLS, autenticação e regras de acesso por tópico.

Dica

Publicar: `publishMessage`.

Qual protocolo usar?

	BLE	USB serial	HTTP na rede local	MQTT via broker
Alcance	~10–30 m	o cabo	a rede Wi-Fi	qualquer lugar com internet
Consumo no sensor	muito baixo	alimentado pelo cabo	alto (Wi-Fi)	médio
Tempo real	sim (notify)	sim	não (*polling*)	sim (push)
Plataformas	Android, iOS	só Android	todas	todas
Vários clientes	um central por vez	um	sim	sim, muitos
Caso ideal	configurar a placa, bancada ao lado	bancada fixa, Arduino sem rádio	painel na rede do laboratório	monitoramento contínuo e alertas

Arquitetura combinada

Use BLE para o *provisionamento* (enviar SSID e senha do Wi-Fi ao ESP32 pelo app) e MQTT para a telemetria do dia a dia.

Arquitetura e referência rápida

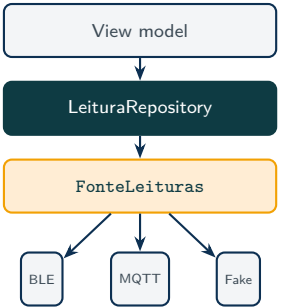
Hardware atrás de uma interface

data/services/fonte_leituras.dart

```dart
abstract interface class FonteLeituras {
  Stream<Leitura> leituras(String bancada);
}

class FonteBle implements FonteLeituras { ... }
class FonteMqtt implements FonteLeituras { ... }
class FonteSerial implements FonteLeituras { ... }

class FonteFake implements FonteLeituras {
  @override
  Stream<Leitura> leituras(String b) => Stream.periodic(
      const Duration(seconds: 1),
      (i) => Leitura(b, 22 + sin(i / 5) * 3));
}
```



Por quê

A UI e as regras não sabem de onde vem o dado. O *fake* permite desenvolver sem placa, demonstrar em sala e testar (Módulo 14).

Referência rápida: pacotes e permissões

Recurso	Pacote	Android (manifesto)	iOS (`Info.plist`)
Câmera e QR code	`camera`, `mobile_scanner`	`CAMERA`	`NSCameraUsageDescription`
Galeria	`image_picker`	nenhuma (Photo Picker)	`NSPhotoLibraryUsageDescription`
Localização	`geolocator`	`ACCESS_FINE_LOCATION`	`NSLocationWhenInUseUsageDescription`
Biometria	`local_auth`	`USE_BIOMETRIC`	`NSFaceIDUsageDescription`
NFC	`flutter_nfc_kit`	`NFC`	`NFCReaderUsageDescription` + capacidade
Notificações	`flutter_local_-notifications`	`POST_NOTIFICATIONS`	pedido em tempo de execução
Bluetooth LE	`flutter_blue_plus`	`BLUETOOTH_SCAN` e `_CONNECT`	`NSBluetoothAlwaysUsageDescription`
Microfone	`record`	`RECORD_AUDIO`	`NSMicrophoneUsageDescription`
Rede e MQTT	`dio`, `mqtt_client`	`INTERNET`	—
Serial USB	`usb_serial`	diálogo do sistema	não suportado

Fechamento

Resumo

Resumo

- Plugins ligam Dart ao nativo; declare permissões e peça no contexto certo.
- Câmera, QR code, GPS, mapas e sensores viram `Futures` e `Streams` em Dart.
- Biometria desbloqueia segredos locais; NFC identifica objetos por aproximação.
- Notificações locais, WorkManager e *foreground services* cobrem o segundo plano.
- ESP32 e Arduino conversam por BLE, USB serial ou MQTT, atrás de uma interface comum.

Glossário (1/2)

Termo	Significado
Plugin	pacote com API em Dart e implementação nativa em cada plataforma
Permissão em tempo de execução	autorização dada pelo usuário num diálogo do sistema
ADC	conversor que transforma uma tensão em número
Taxa de amostragem	quantas medidas são feitas por segundo
Sensor MEMS	sensor micromecânico gravado no silício (acelerômetro, giroscópio)
Fusão de sensores	combinação de sensores para compensar os defeitos de cada um
GNSS	sistemas de navegação por satélite: GPS, GLONASS, Galileo, BeiDou
Trilateração	cálculo da posição a partir das distâncias a pontos conhecidos
Doze	modo que adia rede e alarmes com o aparelho parado

Glossário (2/2)

Termo	Significado
BLE	Bluetooth de baixo consumo, feito para sensores
GATT	organização dos dados BLE em serviços e características
Característica	valor com UUID que pode ser lido, escrito ou notificado
MTU	tamanho máximo de um pacote ATT (23 bytes por padrão)
MQTT	protocolo leve de publicação e assinatura, com *broker*
QoS	nível de garantia de entrega do MQTT: 0, 1 ou 2
UART	comunicação serial assíncrona (formato 8N1, *baud rate*)
NDEF	formato padrão das mensagens em etiquetas NFC

Questões de revisão

1. Descreva o caminho de uma temperatura do sensor do ESP32 até a tela. Onde entram o ADC e a taxa de amostragem?
2. Por que o acelerômetro sozinho não basta para saber a orientação do celular?
3. Quantos satélites o GPS precisa e por quê? Por que, dentro do laboratório, a posição piora?
4. Quando pedir uma permissão, e o que fazer se o usuário negar?
5. Por que o BLE consome tão pouco? O que é o intervalo de conexão?
6. Que QoS do MQTT usar para leituras a cada segundo e para o comando que desliga o relé? Por quê?
7. Explique o formato 8N1 e o que acontece se os dois lados usarem *baud rates* diferentes.
8. Como reduzir o consumo de bateria de um app que monitora a bancada o dia todo?

Exercícios

Exercício 1 — Inventário

Leia o QR code de um equipamento, tire uma foto, registre a posição GPS e compartilhe um resumo.

Exercício 2 — Check-in

Confirme presença no laboratório exigindo distância menor que 50 m **ou** leitura da etiqueta NFC da bancada, e biometria antes de liberar.

Exercício 3 — IoT

Grave o firmware no ESP32, receba a temperatura por BLE e dispare uma notificação local acima de 30 °C.

LabMonitor — Etapa 13

`FonteLeituras` com implementações BLE, MQTT e *fake*; check-in por QR/NFC; alarme sonoro e notificação de temperatura.

Para aprofundar

- Plugins e canais de plataforma: `https://docs.flutter.dev/platform-integration/platform-channels`
- Pacotes Flutter Community (*plus*): `https://github.com/fluttercommunity/plus_plugins`
- permission_handler: `https://pub.dev/packages/permission_handler`
- flutter_blue_plus: `https://pub.dev/packages/flutter_blue_plus`
- ESP32 com Arduino: `https://docs.espressif.com/projects/arduino-esp32`
- MQTT: `https://mqtt.org` mqtt_client: `https://pub.dev/packages/mqtt_client`

Respostas das questões de revisão

1. O sensor transforma temperatura em tensão; o ADC do ESP32 amostra e quantiza em 12 bits; o *firmware* converte em °C e envia por BLE ou MQTT; o app decodifica e exibe. A taxa de amostragem define as leituras por segundo; acima do necessário, só gasta energia.
2. Ele mede a gravidade somada à aceleração do movimento e é ruidoso: com o aparelho em movimento, a inclinação calculada erra. A fusão com o giroscópio (vetor de rotação) resolve.
3. Quatro: três coordenadas e o erro do relógio do receptor. Dentro de prédios, o sinal é bloqueado ou refletido e o sistema recorre a Wi-Fi e torres, menos precisos.
4. No momento em que o recurso é usado, explicando o motivo quando não for óbvio. Se o usuário negar, o app segue funcionando sem o recurso e oferece um atalho para as configurações.
5. O rádio fica quase sempre desligado e só acorda nos eventos de conexão. O intervalo de conexão é o tempo entre esses encontros, de 7,5 ms a 4 s.
6. Leituras: QoS 0 ou 1, pois uma perdida logo é substituída pela seguinte. Relé: QoS 1 com comando idempotente (ou QoS 2), pois perder o comando é grave e repeti-lo não pode causar dano.
7. Um bit de início, 8 de dados, nenhum de paridade e 1 de parada. Com *baud rates* diferentes, o receptor amostra nos instantes errados e lê lixo.
8. MQTT ou *push* em vez de *polling*, envios agrupados, taxa de amostragem menor, *notify* no BLE, o pacote `workmanager` para tarefas periódicas e nada de GPS contínuo.

Fim do Módulo 13

Próximo: Módulo 14 — Testes: unidade, widget, *golden* e integração.